Cyber Physical Systems

Basics of TinyML

Sambit Sahoo
Intern, CoDA Lab

Under the guidance of
Dr. Sudip Roy, Dept. of CSE, IIT Roorkee

sambit22@iiserb.ac.in
GitHub Link

June 13, 2025

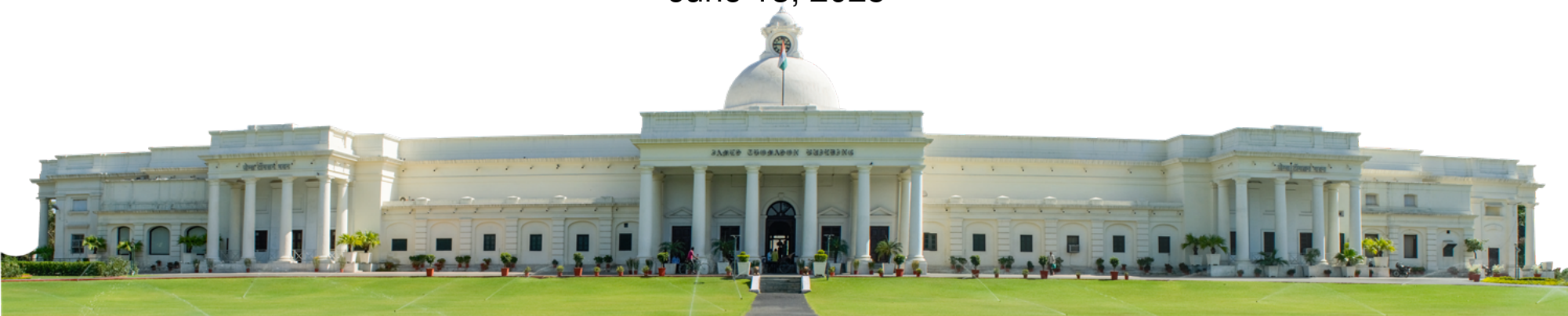


Table of Contents

Overview of Machine Learning

- **What is Learning?**
  Learning is the act of acquiring knowledge over experience to improve performance.

- **Machine Learning**
  ML is the branch of data science that enables computer systems to learn patterns and make predictions from the data without being explicitly programmed for each specific task.

  Machine learning problems broadly divided into:
  1. Supervised learning
  2. Unsupervised learning

Supervised & Unsupervised Learning

- **Supervised Learning**
  A supervised learning algorithm analyzes the previously labeled data and produces an inferred function, which can be used for mapping new instances.

  Let $(\vec{X}_i, y_i),\ \forall i = 1, 2, \ldots, n$, be the instances in a dataset, where $\vec{X}_i = [x_{i1}, x_{i2}, \ldots, x_{im}] \in \mathbb{R}^m$ is an m-dimensional feature vector and n is the number of instances (also called data points). Here, $x_{ij},\ \forall j = 1, 2, \ldots, m$, are the individual features of the i^{th} data point, and y_i is the corresponding target variable for $\vec{X}_i$, $\forall i$.
  The aim is to build a model using this dataset to estimate the target variable of a new instance, say, $\vec{X}_0 \in \mathbb{R}^m$.

- Broadly classified into *Classification* and *Regression* problems.

Supervised & Unsupervised Learning

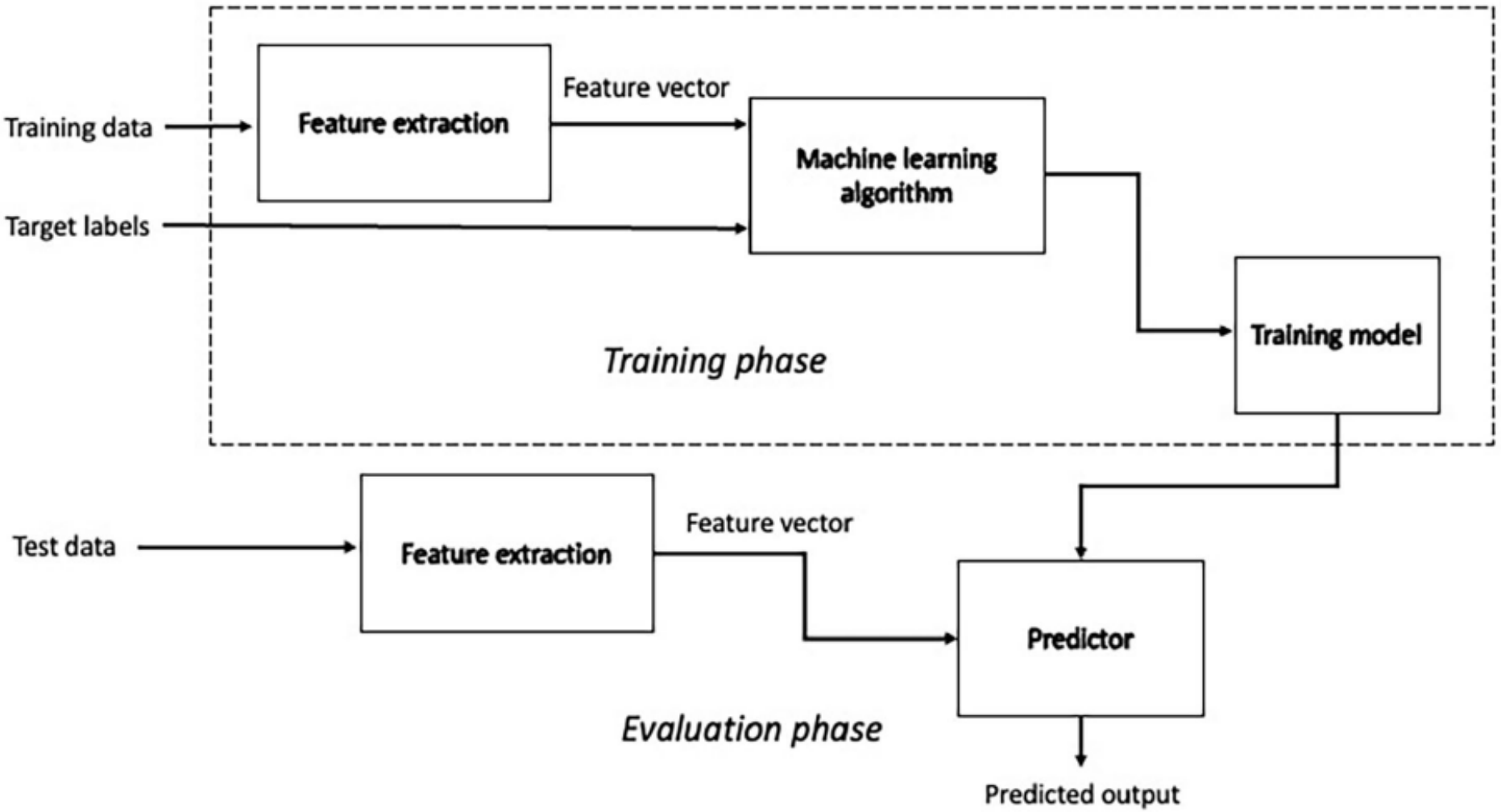
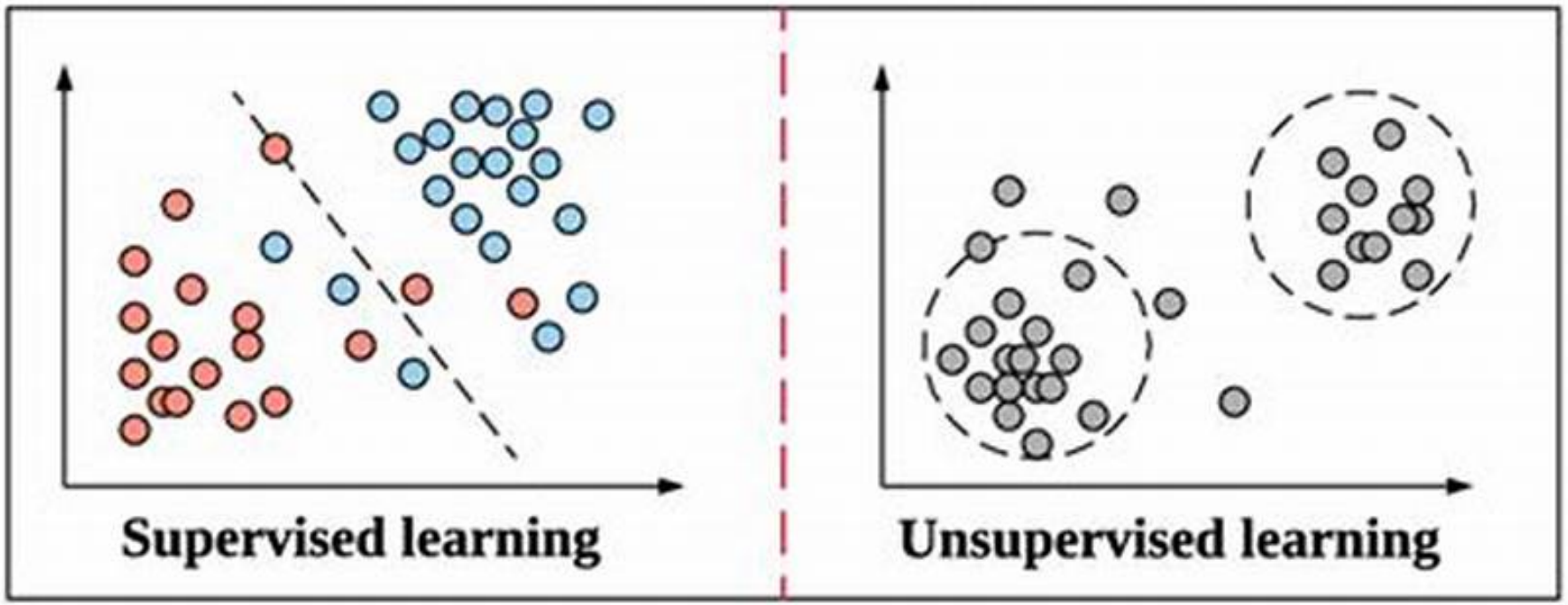


Figure 1.1: Block diagram of supervised learning approach

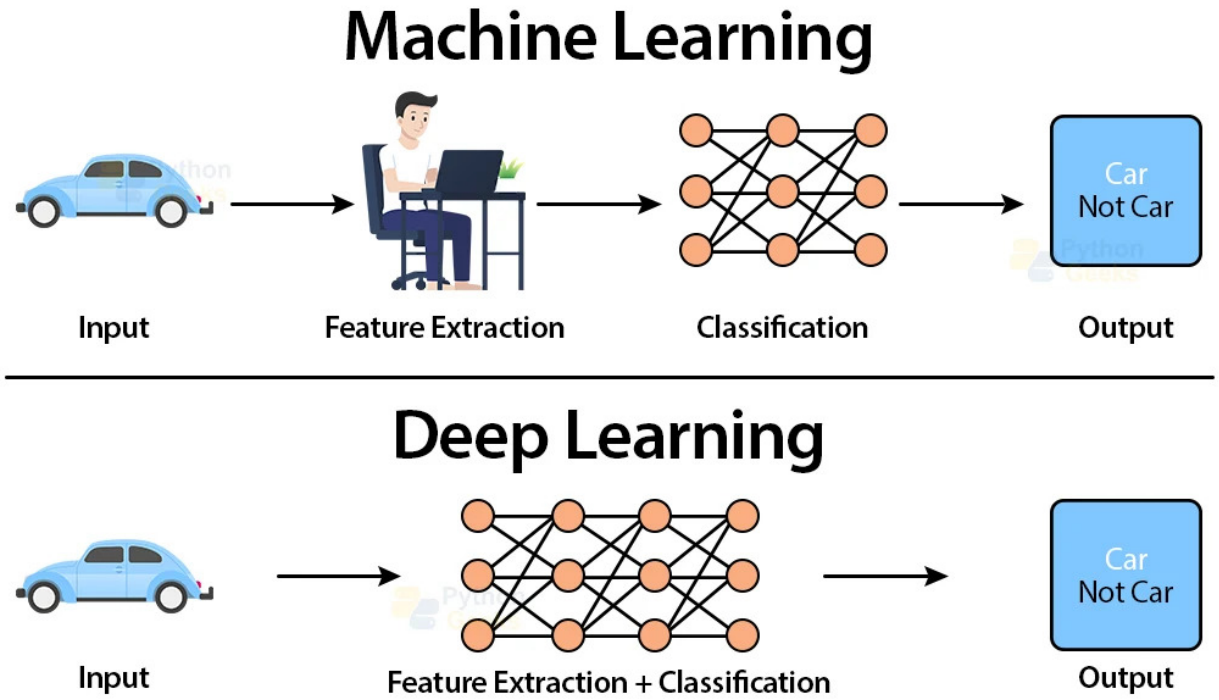
Supervised & Unsupervised Learning

- **Unsupervised Learning**
  In the unsupervised learning, no class labels are available in the given data to build the model. The objective is to find natural groups or clusters of similar objects within the data.
- Broadly divided to *Clustering* and *Density Estimation* problems.



Deep Learning

- ML algorithms cannot learn directly from raw unstructured data.
- DL algorithms can directly take raw data as input and can extract the relevant features automatically, therefore bypassing the need for manual feature extraction.



- Computationally expensive, Needs (**GPU**) or (**TPU**).

Artificial Neural Network (ANN)

- The layered structure of Artificial Neural Network (ANN) is inspired by the biological nervous system.
- An ANN comprises of neurons or nodes in a layered structure where each layer is connected to another layer to analyze complex patternsand relationships in data.

Q. How does an ANN work ?

- **Forward Propagation:**
  - Inputs are multiplied by weights and summed.
  - The result is passed through activation functions layer by layer.
- **Loss Calculation:**
  - The predicted output is compared with the actual value.
  - A *loss function* (MSE, cross-entropy) is used to quantify the error.
- **Backpropagation:**
  - Computes gradients of the loss with respect to each weight.
  - Updates weights using *gradient descent* to minimize the loss.

Artificial Neural Network (ANN)

- Building Sequential Model by adding layers through ".add()"

```
from tensorflow.keras.layers import Dense
model = Sequential()
model.add(Dense(512, activation = 'relu', input_shape = (784,)))
model.add(Dense(128, activation = 'relu'))
model.add(Dense(num_class, activation = 'softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam',metrics=['accuracy'])
model.summary()
```

[18]

```
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
dense_12 (Dense)	(None, 512)	401,920
dense_13 (Dense)	(None, 128)	65,664
dense_14 (Dense)	(None, 10)	1,290

```
Total params: 468,874 (1.79 MB)

Trainable params: 468,874 (1.79 MB)

Non-trainable params: 0 (0.00 B)
```

Artificial Neural Network (ANN)

- Training the model & Finding accuracy on test data.

```
model.fit(X_train, y_train, batch_size=128, epochs=10, verbose=1)
```

```
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - accuracy: 0.8751 - loss: 0.4318
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9732 - loss: 0.0897
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9822 - loss: 0.0565
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9879 - loss: 0.0391
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9916 - loss: 0.0279
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9944 - loss: 0.0177
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - accuracy: 0.9956 - loss: 0.0145
```

```
score = model.evaluate(X_test, y_test)
print('loss on test data: ', score[0])
print('accuracy on test data:', score[1])
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9784 - loss: 0.0932
loss on test data:  0.07661768794059753
accuracy on test data: 0.9821000099182129
```

Convolutional Neural Network (CNN)

What is a CNN?

- A type of deep learning model specifically designed to process grid-like data such as images.
- Mimics the visual perception mechanism of the human brain.
- Automatically learns spatial hierarchies of features through layers.
- Commonly used in image classification, object detection, and medical image analysis.

Why CNN for Images?

- Preserves spatial structure of the data.
- Reduces the number of parameters compared to fully connected networks.
- Learns relevant features automatically using filters.

Convolutional Neural Network (CNN)

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dropout, Dense

model = Sequential()
model.add(Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape = (28,28,1)))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Conv2D(64, kernel_size=(3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dropout(0.5))
model.add(Dense(num_class, activation='softmax'))
model.summary()
```

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_4 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_5 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_5 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_2 (Flatten)	(None, 1600)	0
dropout_2 (Dropout)	(None, 1600)	0
dense_2 (Dense)	(None, 10)	16,010

CNN Model Architechture

Model Layers and Their Functions

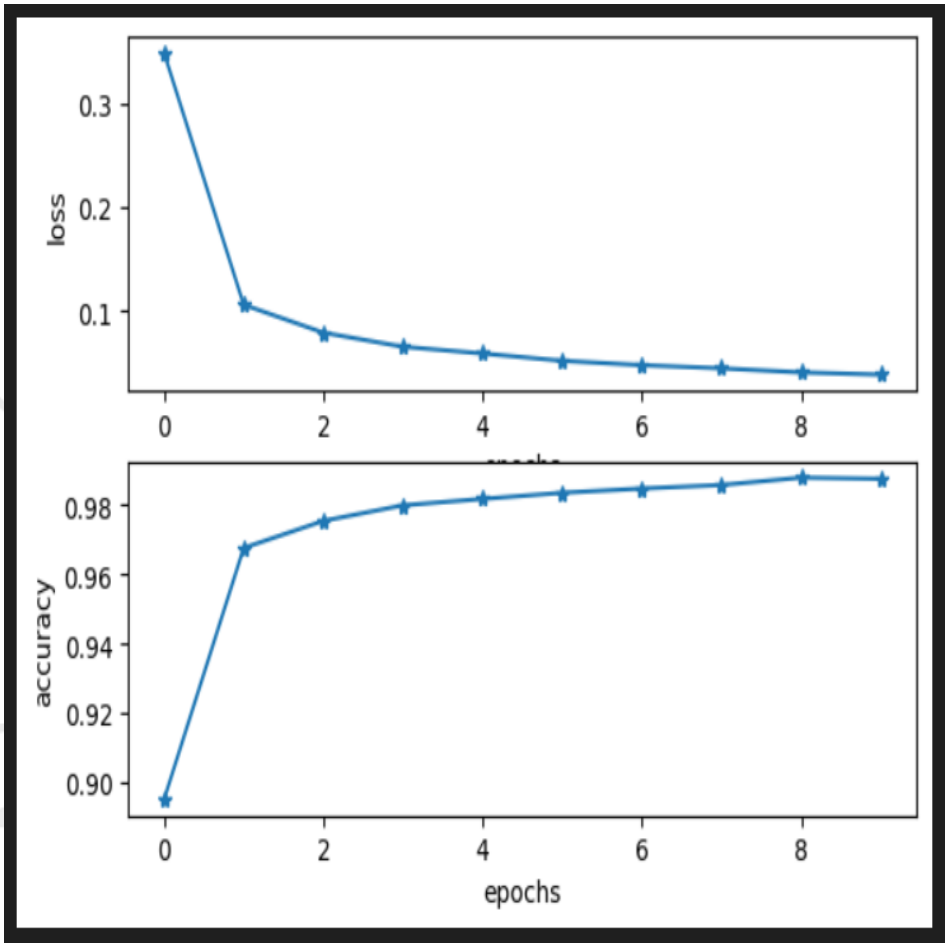
- **Input Layer:** Accepts input images.
- **Conv2D (32 filters, 3×3):** Applies convolution to input locally to extract features.
- **MaxPooling2D (2×2):** Reduces the spatial dimensions (downsamples) while keeping important features.
- **Flatten:** Converts the 2D feature maps into a 1D vector for feeding into the dense layer.
- **Dropout (rate = 0.5):** Randomly deactivates 50% of neurons during training to prevent overfitting.
- **Dense (Softmax):** Outputs probabilities for each class using the softmax activation function.

Convolutional Neural Network (CNN)

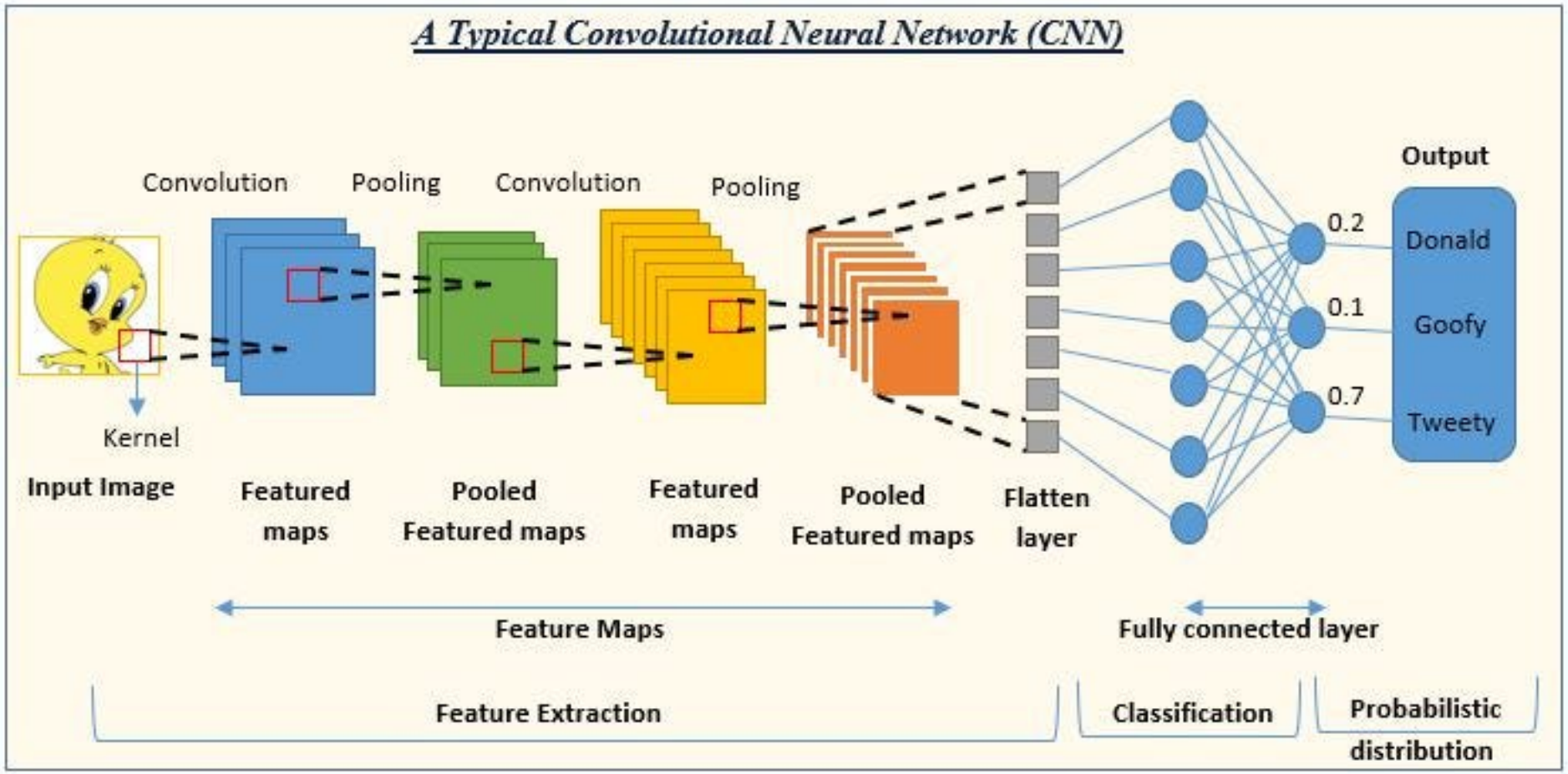
```
score = model.evaluate(X_test, y_test)
print('loss on test data: ', score[0])
print('accuracy on test data:', score[1])
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9904 - loss: 0.0309
loss on test data:  0.025411060079932213
accuracy on test data: 0.9922999739646912
```

- CNN achieves 99% test accuracy, slightly outperforming the 98% from the feedforward ANN on the MNIST dataset.
- Image in the right shows the plot of loss and accuracy w.r.t. epochs during training of CNN repectively.



CNN Workflow

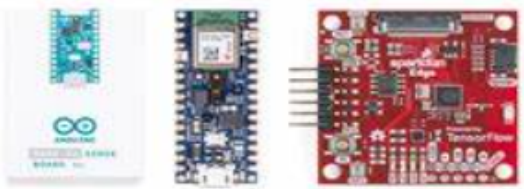


TinyML

Q. Why TinyML?

- Large deep learning algorithms typically run on powerful desktops, data centers, or cloud servers.
- Smaller edge devices and microcontrollers are severely resource-constrained (e.g., 256KB memory and 64MHz CPU).
- TinyML brings ML inference to low-power, low-memory devices.
- Enables real-time, offline, and privacy-preserving applications on the edge.

TensorFlow Lite



- With TensorFlow Lite Converter, we can convert a TensorFlow model into a compressed **FlatBuffers** format that is optimized for mobile and edge devices.

TinyML

After we make an NN model, we save the baseline model by:

```
model.save('baseline_model.h5')
```

Converting to TFLite:

```
from tensorflow.keras.models import load_model
baseline_model = load_model('baseline_model.h5')

converter = tf.lite.TFLiteConverter.from_keras_model(baseline_model)
tflite_model = converter.convert()
```

Export model as a concrete function:

```
func = tf.function(baseline_model).get_concrete_function(tf.TensorSpec(baseline_model.inputs[0].shape, baseline_model.inputs[0].dtype))
func.graph.as_graph_def()               # serialized graph representation of the concrete function

converter = tf.lite.TFLiteConverter.from_concrete_functions([func])  # converting the concrete function to TFLite
tflite_model = converter.convert()
```

TinyML

Applying the TFLite model by ".invoke()":

```
tflite_model_file = 'tflite_models/model.tflite'
interpreter = tf.lite.Interpreter(model_path = str(tflite_model_file))

interpreter.allocate_tensors()

input_index = interpreter.get_input_details()[0]['index']
output_index = interpreter.get_output_details()[0]['index']

pred_list = []
for images in X_test:
    input_data = np.array(images, dtype=np.float32)
    input_data = input_data.reshape(1, input_data.shape[0], input_data.shape[1], 1)

    interpreter.set_tensor(input_index, input_data)
    interpreter.invoke()     #applies the tflite model

    prediction = interpreter.get_tensor(output_index)
    prediction = np.argmax(prediction)
    pred_list.append(prediction)
```

TinyML Workflow

- First, we create an instance of the Interpreter class. It takes the path containing the .TFLITE file as an input.
- Allocate memory to the Interpreter by calling the function allocate_tensors().
- Use `details()` to inspect input and output tensors.
- Serialization converts the model graph into a portable format (e.g., `.pb` file) that can be:
  - Saved to disk
  - Shared with others
  - Re-loaded later without redefining the model
- Get an image from test data and reshape it to match the model's expected input shape.
- Set the input tensor using `set_tensor()`.
- Call `Interpreter.invoke()` to run inference.
- Retrieve the output tensor's value.
- Convert the output into the predicted class label.

Baseline Model v/s TFLite

```
>>>> Baseline Model Accuracy: 91.80%
>>>> Baseline Model Prediction Time: 1.0991 seconds
>>>> Baseline model size: 2147.92 KB (2.10 MB)

>>>> TFLite Model Accuracy: 91.80%
>>>> TFLite Model Prediction Time: 0.7787 seconds
>>>> TFLite model size: 706.09 KB
```

- Here we can check that the prediction time and model size are reduced without significant change in Accuracy for MNIST dataset. Hence it is helpful in case of microcontrollers.

References

- **GitHub:** Link for GitHub
- **Book:** Rohan Banerjee, "Hands-on TinyML"
- **Coding Platform:** VS-Code

Thanks.